

School of XXX, University XXX, City, Post Code,
China. E-mail: corresponding@email.cn.

Cross-Domain Differentiable Monte Carlo Ray Tracing with Embedded Physics Proxies for Heliostat Surface Optimization

First A. Author¹, Second B. Author², and Third C. Author³

© The Author(s)

Abstract We present a GPU-native differentiable Monte Carlo ray tracing (DMCRT) system that embeds a finite-element-derived thin-plate spline (TPS) proxy model and a 20-bin nonlinear gravity deformation library directly into the ray tracing shaders, enabling the optical loss gradient to back-propagate through both refraction optics and mechanical deformation simultaneously. The renderer runs entirely on GPU with zero readback per iteration: a collaborative binary search computes the S95 spillage threshold on-device, a single Vulkan submit batches 10+ compute dispatches, and only 4 bytes of scalar loss leave the GPU per iteration. Through a systematic investigation of gravity-induced deformation mechanisms using this differentiable framework, we design a gradient audit methodology that decomposes the deformation field into spectral bands, projects each band onto the bolt influence subspace, and quantifies the structurally irreducible floor—the fraction of the optical penalty that no bolt adjustment strategy can remove under a given support layout. Applied to a 20-mirror heliostat field at Delingha (37.36°N, 97.29°E), the system reveals that gravity-induced S95 degradation reaches $1.001 \times -1.539 \times$ with strong spatial non-uniformity predictable from site geometry alone, and that 84–87% of the correctable gap is structurally irreducible under a fixed 35-bolt support layout. Extending the optimization variable from bolt strokes to the structural layout itself—by making the support edge margin an optimization variable—yields a 26.7% annual S95 reduction with zero additional manufacturing cost. The framework transforms heliostat surface design from empirical per-mirror adjustment into a bounded, certifiable, and fully automated differentiable workflow.

Keywords differentiable rendering, Monte Carlo ray tracing, physics-based modeling, GPU computing, heliostat optimization, structural-optical co-design

1 Introduction

Differentiable rendering has matured rapidly in computer graphics over the past decade, with systems such as Mitsuba 2/3 [1, 2], ZeroGrads [3], and DiffRP [4] enabling gradient-based optimization of scene parameters through physically-based light transport. The core idea—exposing exact or approximate gradients of pixel values with respect to scene parameters—has proven transformative for inverse problems in appearance modeling, geometry reconstruction, and computational design [5].

A parallel development is unfolding in concentrating solar power (CSP) engineering, where differentiable Monte Carlo ray tracing (DMCRT) has recently been applied to heliostat aiming optimization [7], canting angle optimization [8], and in-situ surface metrology [9–11]. These works share a common characteristic: the surface being rendered is *given*—represented either as a fixed geometric primitive, such as an ideal elliptic paraboloid, or parameterized by purely optical degrees of freedom (panel tilt angles, focal lengths). In all cases, the surface shape is treated as a purely geometric entity. The mechanical degrees of freedom that actually *produce* the surface shape (support bolt displacements) and the physical loads that *distort* it (gravity) have never been connected to the optical objective through a differentiable gradient chain.

1.1 Motivation: Why Physics Proxies Matter for Heliostat Optimization

The limitations of purely geometric surface representations become evident when examining the physical reality of a heliostat mirror. A large-span thin glass mirror (12.84×9.45 m, 4 mm thick) supported by discrete bolts sags under its own weight by several millimeters, and this deformation field changes continuously as the mirror tracks the sun throughout the day and year. The resulting slope errors—on the order of several milliradians—are of the same magnitude as the sun’s intrinsic width (~ 2.2 mrad for the Buie model at CSR 0.01),

meaning that gravity-induced deformation directly competes with the optical signal the mirror is designed to concentrate.

Current approaches to modeling surface slope errors in CSP ray tracing fall into two categories. The first treats slope error as a spatially uniform Gaussian random variable with a single scalar parameter σ_s , which convolves the ideal reflected ray distribution [22, 23]. This approach is computationally efficient but cannot capture the spatially structured, tilt-angle-dependent deformation patterns produced by gravity. The second approach, employed in coupled structural-optical optimization [15–18], uses FEA to compute the actual deformed surface, then passes it to a separate ray tracing simulation. While physically accurate, the serial, offline nature of this coupling restricts optimization to parameter sweeps or black-box evolutionary algorithms over a handful of design variables, with each evaluation requiring a complete FEA solve plus a complete ray tracing simulation.

Neither approach provides what is needed for gradient-based surface design: a physics model that is *accurate enough* to capture the dominant deformation modes, *differentiable* to enable gradient-based optimization, and *fast enough* to evaluate inside an iterative rendering loop. This is the gap that an embedded physics proxy aims to fill.

At the same time, embedding physics into a differentiable renderer raises a deeper question: *how do we know whether the embedded physics is actually contributing to the optimization, rather than merely existing in the forward graph?* The question is not academic—the history of physics-informed machine learning contains cases where a physics term in the loss function or the forward model was functionally inert, because its gradient contribution was numerically dominated by other terms or structurally prevented from reaching the design variables. Answering this question requires dedicated analytical tools that decompose the physics contribution and quantify what is reachable by the optimizer and what is not.

1.2 Research Questions and Contributions

These considerations motivate three research questions that structure this paper:

- **RQ1 (Magnitude):** What is the real annual optical penalty caused by gravity-induced deformation, and how does it distribute across a heliostat field?
- **RQ2 (Compensability):** For a fixed support structure, what fraction of the gravity penalty is *physically removable* by bolt adjustment—i.e., where is the floor that separates any reachable design from the gravity-free optimum?

- **RQ3 (Structural optimization):** Can we move the floor itself by making structural layout parameters—support margin, bolt density, material thickness—optimization variables, rather than merely compensating within a fixed structure?

To answer these questions, we present a complete GPU-native DMCRT system that embeds a finite-element-derived TPS proxy and a 20-bin NLGEOM gravity library directly in the ray tracing shaders, enabling end-to-end gradient-based optimization across both the optical and mechanical domains. Our contributions are:

- (1) **A zero-readback GPU differentiable rendering pipeline** with collaborative on-device S95 threshold search (20 iterations of binary search, single workgroup, only 4 B of scalar loss read back per iteration), single-submit batching of 10+ compute dispatches, compile-time sunshape specialization, and per-ray pre-culling (§3).
- (2) **A cross-domain gradient chain** that propagates optical loss gradients through double-refraction glass optics *and* a finite-element-derived thin-plate spline (TPS) mechanical proxy, mapping surface flux gradients to 35 independent bolt stroke gradients in a single GPU dispatch sequence (§4).
- (3) **A gradient audit methodology** that decomposes the gravity deformation field into affine, quadratic, and higher-order bands, projects each band onto the bolt influence subspace, and quantifies the *structurally irreducible floor*—the fraction of the optical penalty that no bolt adjustment strategy can remove under the given support layout (§5).
- (4) **A two-level design path** from surface compensation to structural optimization: discovering that 80–90% of gravity slope energy concentrates in the cantilevered edge flaps (not between bolts), deriving a margin–span conservation law that predicts an interior optimum for the support margin, and achieving a **26.7% annual S95 reduction** with zero additional manufacturing cost (§6).
- (5) **An application case study** on a 20-mirror heliostat field at Delingha, China, providing the first complete gravity penalty map, a geometric tilt-angle distribution model that predicts penalty non-uniformity from site latitude and tower geometry alone, and experimental evidence that grid topology dominates per-bolt position micro-tuning (§7).

2 Related Work

2.1 Differentiable Rendering Systems

The modern differentiable rendering stack was established by Mitsuba 2 [1], which introduced retargetable automatic differentiation through a symbolic computation graph over physically-based light transport. Mitsuba 3 [2] extended this with GPU-accelerated wavefront differentiable path tracing, enabling optimization over object geometry, materials, and lighting at practical scales. ZeroGrads [3] demonstrated that careful engineering of the AD overhead can make differentiable rendering competitive with forward-only rendering for production assets.

Our system occupies a distinct point in the design space. Rather than implementing a general-purpose differentiable renderer with a symbolic computation graph, we build a *hardware-specialized* pipeline where the entire gradient computation is written in Slang [6] with explicit `bwd_diff` annotations, compiled to SPIR-V, and dispatched on a single Vulkan queue with zero CPU–GPU synchronization within an iteration. This design choice trades generality for throughput: each iteration processes multiple sun directions (sampled from an annual distribution) in a single `vkQueueSubmit` call, with only the final scalar loss value crossing the PCIe boundary. The collaborative on-device S95 binary search (§3.2) eliminates what would otherwise be the dominant readback bottleneck.

2.2 Differentiable MCRT in Concentrating Solar Power

Several recent works have applied differentiable ray tracing to CSP problems. Pargmann et al. [9] used DMCRT for *in-situ* heliostat metrology, reconstructing surface error fields from calibration images. Inverse deep learning ray tracing [10, 11] predicts heliostat surfaces from focal spot measurements with sim-to-real transfer. Canting optimization via differentiable ray tracing [8] adjusts facet tilt angles to minimize spillage, and aiming optimization via DMCRT [7] optimizes heliostat pointing strategies at the field level.

These works validate that DMCRT gradients are well-behaved and useful in CSP optics. However, they all treat the surface as either a given geometric primitive or parameterized by purely optical variables (tilt, focal length). The mechanical actuators that physically produce the surface (support bolts) and the structural loads that physically deform it (gravity) are absent from the differentiable graph. Our contribution is to close this loop: bolt displacement → TPS surface deformation → ray tracing → S95 loss → gradient back to bolts, with

gravity entering both the surface height *and* the normal vectors at every step.

2.3 Physics Proxies in Differentiable Pipelines

The broader trend of embedding physical models into differentiable optimization has been termed physics-informed machine learning [12] and has seen applications from structural optimization [13] to wind farm digital twins [14]. The dominant paradigm trains a learned surrogate (neural network) to approximate the physical simulator, then differentiates through the surrogate. Our approach is different: we use an *analytic, hand-derived* proxy (thin-plate splines with a partition-of-unity property) whose gradients are exact and whose error budget can be bounded by direct comparison with finite-element solutions (§4.4). This trades the surrogate’s flexibility for the proxy’s auditability—a choice that proves essential for the gradient audit in §5.

2.4 Coupled Structural-Optical Heliostat Optimization

In the heliostat engineering literature, connecting structural design to optical performance has been explored through offline, serial coupling: FEA computes the deformed surface, then a separate ray tracing pass evaluates the optical consequence [15–18]. These works established the important baseline that structural parameters (bolt spacing, margin, material) significantly affect optical quality, and that the relationship is non-monotonic (an interior optimum exists for bolt spacing). However, their serial nature restricts optimization to parameter sweeps or black-box evolutionary algorithms over a handful of design variables, with each evaluation requiring a complete FEA solve plus a complete ray tracing simulation.

Our framework differs in three respects: (i) the gradient chain makes the entire system end-to-end differentiable, enabling gradient-based search over continuous parameter spaces; (ii) the on-device GPU architecture makes each evaluation cheap enough for optimization loops (minutes, not hours); and (iii) the differentiability enables *gradient-reachable subspace analysis*—answering not just “which parameter combination is better” but “what is the physical upper bound of any adjustment strategy under this layout” (§5). Table 1 positions our work against the closest prior art.

3 System Architecture

Our differentiable renderer is built on Vulkan compute shaders written in Slang [6], which provides automatic differentiation via `[Differentiable]` type annotations and `bwd_diff` reverse-mode code generation. The key architectural decisions

Table 1 Positioning against closest prior work.

Work	Task	Design variables	Structure model	E2E diff.	Bounds analysis
[7]	Aiming opt.	2 DOF/mirror	None	Yes	No
[8]	Canting opt.	Panel tilts	None	Yes	No
[9]	In-situ metrology	Surface normals	None	Yes	No
[10, 11]	Surface prediction	Surface coeffs.	None	Neural	No
[15, 16]	Support $\rightarrow$ optics	Spacing d , count N	FEA (offline)	No	No
[17, 18]	Structural-optical	Structural dims.	FEA + ray trace	No	No
This work	Surface design	35 bolt strokes + margin	TPS + 20-bin NLGEOM gravity	Yes	Yes

are driven by a single constraint: *every optimization iteration must process multiple sun directions at interactive rates, with the full gradient chain crossing both optical and mechanical domains.*

3.1 Single-Submit Multi-Dispatch Pipeline

The optimization loop iterates over an annual set of sun directions $\mathcal{D} = \{(s_d, \text{DNI}_d)\}_{d=1}^{|\mathcal{D}|}$ (typically 36–334 directions, §??). For each iteration, a *single vkQueueSubmit* call batches all dispatches for all sun directions:

- (1) **Mechanical forward** (`computeBoltSurface`, 1 workgroup): TPS influence superposition + 20-bin gravity interpolation $\rightarrow 32 \times 32$ surface height field $\mathbf{y}$ and normal field $\mathbf{n}$.
- (2) **Ray tracing forward** (`renderForward`, k tiles $\times \sim 3950$ groups): Monte Carlo sampling from receiver pixels to heliostat grid points, double-refraction through 4 mm glass ($n = 1.523$), Buie sunshape evaluation, flux accumulation in a 157×50 receiver image.
- (3) **GPU S95 search** (`computeS95FindLevel`, 1 workgroup): Collaborative binary search for the flux threshold containing 95% of total energy (§3.2).
- (4) **Loss + backward optics** (`computeS95LossBUF` + `renderBackwardBolt`): Sigmoid loss gradient w.r.t. flux $\rightarrow$ `bwd_diff` through refraction to surface height/normal gradients $\partial L / \partial(y, y_u, y_v)$ per grid point.
- (5) **Mechanical backward** (`reduceSurfaceGradients` + `projectBoltGradients`): Cross-group reduction of fixed-point gradient tiles $\rightarrow$ projection onto bolt gradients $\partial L / \partial h_b$ through TPS influence functions.
- (6) **Adam update** (`boltAdamStep`, 1 workgroup): Tanh-bounded parameter update in unbounded ε -space, with optional regularization terms (anchor, bend, soft stroke wall).

Table 2 GPU dispatches per sun direction per iteration (single submit).

Dispatch	Shader	Grid	Pu
<code>computeBoltSurface</code>	<code>bolt_forward</code>	(1, 1, 1)	Mech
<code>clearFlux</code>	<code>forward</code>	(1, 1, 1)	Zero f
<code>renderForward</code>	<code>forward (A2 typed)</code>	(k , 3950, 1)	MCR
<code>finalizeFlux</code>	<code>forward</code>	(1, 1, 1)	Flux no
<code>computeS95FindLevel</code>	<code>s95_gpu</code>	(1, 1, 1)	S95 bir
<code>computeS95LossBUF</code>	<code>s95_gpu</code>	(10, 4, 1)	Loss +
<code>renderBackwardBolt</code>	<code>bolt_backward (A2)</code>	(3950, k , 1)	MCRT
<code>reduceSurfaceGradients</code>	<code>bolt_backward</code>	(1, 1, 1)	Cross-g
<code>projectBoltGradients</code>	<code>bolt_backward</code>	(1, 1, 1)	$\rightarrow \partial$
<code>boltAdamStep</code>	<code>bolt_optimizer</code>	(1, 1, 1)	Adan

Table 2 summarizes the per-iteration GPU dispatches. The key engineering property is that *no intermediate buffer is read back to the CPU during the optimization loop*: flux images, S95 thresholds, and surface gradients all remain device-resident, communicated between dispatches via pipeline barriers. Only the final scalar loss (4 bytes, fixed-point $\times 10^3$) and per-mirror S95 values cross the PCIe boundary.

3.2 GPU-Resident S95 Threshold Search

The S95 metric (the area of the smallest contiguous set of receiver pixels containing 95% of intercepted energy [20]) serves as the annual-averaged optical loss. Computing it requires finding a flux threshold τ^* such that $\sum_{p: f_p > \tau^*} f_p = 0.95 \cdot E_{\text{total}}$.

A naive implementation would read back the entire flux image ($157 \times 50 = 7,850$ floats), sort on the CPU, and perform the binary search—adding ~ 30 kB of PCIe traffic per sun direction per iteration (~ 1.1 MB for 36 suns, ~ 10 MB for 334 suns, per iteration). We instead implement the search entirely on the GPU as a single-workgroup collaborative algorithm:

- (1) **Energy scan**: Each of 256 threads scans its assigned flux range to compute the group-wide total energy E_{total}

and maximum flux $f_{\max}$, using wave-level reductions.

- (2) **Binary search (20 iterations):** The workgroup maintains a search interval $[f_{\text{lo}}, f_{\text{hi}}]$, initially $[0, f_{\max}]$. Each iteration sets the probe threshold $f_{\text{mid}} = (f_{\text{lo}} + f_{\text{hi}})/2$, then each thread accumulates energy above f_{mid} within its range. A single `WorkGroupAllReduce` determines whether the accumulated fraction exceeds 0.95, and the interval is halved accordingly.
- (3) **Result:** After 20 iterations, τ^* is stored to `s95State[0]` (a 4-float device buffer, never read back). The total energy is stored to `s95State[2]` for use by the energy efficiency term (§3.4).

The 20-iteration depth matches a single-precision float mantissa (23 bits), ensuring convergence to floating-point precision. The relative error compared to a CPU reference implementation is $\sim 10^{-6}$, limited only by floating-point reduction order. An environment variable `BEZIER_S95_GPU=0` can restore the CPU readback path for A/B validation.

3.3 Sunshape Specialization and Ray Culling

Two further engineering optimizations are worth noting:

A2 Compile-time sunshape dispatch (forward.slang):

Three specialized ray tracing entry points (`renderForwardBuie`, `renderForwardPillbox`, `renderForwardGaussian`) are produced by Slang’s generic specialization with a `let SUNSHAPE_TYPE` compile-time constant. The C++ dispatcher selects the matching pipeline at runtime based on the configuration’s `sun_type` field. This eliminates a per-ray branch on the sunshape model, which accounts for $\sim 5\%$ of inner-loop instructions.

A1 Per-ray pre-culling (`ray_cull`, default on): Before generating Box-Muller Gaussian samples for the sunshape convolution, each ray tests $\cos(\text{reflected_dir}, \text{sun_dir}) \geq \cos(\theta_{\text{support}} + \delta_{\text{margin}})$, where $\theta_{\text{support}} = 0.0436$ rad (Buie cutoff) and δ_{margin} is a configurable margin (default 8 mrad). Rays failing this test are marked invalid via a bitmask and skipped in both forward and backward passes. At the North 300 m mirror, this reduces total iteration time by **4.8%** with exact bitwise agreement to the unculled reference.

3.4 Regularization Suite

The total loss function combines the S95 sigmoid loss with optional regularization terms, all implemented with analytic gradients in the GPU Adam shader:

$$\mathcal{L}(\mathbf{h}) = \underbrace{\frac{1}{|\mathcal{D}|} \sum_d \text{S95}(\mathbf{h}; \mathbf{s}_d)}_{\text{S95 sigmoid}} + \underbrace{\lambda_E \left(\frac{E_{\text{ref}}}{E} - 1 \right)^2}_{\text{energy efficiency}} + \underbrace{\lambda_s (\mathbf{h} - \mathbf{h}^*)^\top \mathbf{G} (\mathbf{h} - \mathbf{h}^*)}_{\text{slope-space anchor}} \quad (1)$$

The bolt heights are parameterized as $h_b = h_{\max} \tanh(\varepsilon_b)$ (tanh-bounded, L4), with Adam updates performed in the unbounded ε -space. The chain-rule factor $dh_b/d\varepsilon_b = h_{\max}(1 - \tanh^2 \varepsilon_b)$ is folded into the gradient, and the learning rate is compensated as $\text{lr}_\varepsilon = \text{lr}/h_{\max}$ so that the physical step size near $\varepsilon = 0$ (where $h \approx 0$) matches the lr in both bounded and unbounded modes.

The energy efficiency term λ_E prevents the optimizer from “cheating” at large slant ranges (≥ 900 m) by spilling most rays off the receiver to achieve a spuriously small S95. Setting $\lambda_E \geq 0.5$ eliminates this behavior while preserving genuine surface improvements at short ranges (≤ 600 m).

4 Physics Proxy and Cross-Domain Gradient Chain

The surface model in the plate-local coordinate frame (plate normal = y , plate plane = x - z) decomposes the normal displacement field into a gravity term and a bolt superposition term:

$$w(\mathbf{r}; \theta, \mathbf{h}) = w_{\text{grav}}(\mathbf{r}; \theta) + \sum_{b=1}^{N_b} h_b \cdot \phi_b(\mathbf{r}) \quad (2)$$

where w_{grav} is the zero-bolt pure-gravity displacement from NLGEOM finite-element analysis (20 tilt-angle bins, bilinearly interpolated), and ϕ_b is the unit-displacement influence function of bolt b , constructed via thin-plate spline (TPS) interpolation.

4.1 TPS Influence Functions

For each bolt b , we solve the TPS linear system:

$$\begin{bmatrix} \mathbf{K} & \mathbf{P} \\ \mathbf{P}^\top & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{c}_b \\ \mathbf{d}_b \end{bmatrix} = \begin{bmatrix} \mathbf{e}_b \\ \mathbf{0}_3 \end{bmatrix}, \quad K_{ij} = r_{ij}^2 \log(r_{ij}^2), \quad \mathbf{P} = [\mathbf{1}, \mathbf{X}, \mathbf{Z}] \quad (3)$$

with Tikhonov regularization $\lambda = 10^{-6}$ on the diagonal $K_{ii} = \phi(0) + \lambda$. The full-plate response $\phi_b(x, z) = \sum_j c_{b,j} \cdot r_j^2 \log(r_j^2) + d_{b,0} + d_{b,1}x + d_{b,2}z$ satisfies the **partition of unity** property $\sum_b \phi_b(\mathbf{r}) \equiv 1$ (peak violation $< 1.3 \times 10^{-6}$), which guarantees physically correct linear superposition of bolt effects.

The system is 38×38 for $N_b = 35$ bolts (35+3 polynomial terms), condition number $\sim 4.2 \times 10^6$, solved by dense LU

in Python (< 1 s). First-order derivatives are computed analytically:

$$\frac{\partial \phi_b}{\partial u} = \frac{\partial \phi_b}{\partial x} \cdot W, \quad \frac{\partial \phi_b}{\partial v} = \frac{\partial \phi_b}{\partial z} \cdot L \quad (4)$$

where $W = 12.84$ m, $L = 9.45$ m are the plate dimensions, consistent with the shader’s tangent-vector convention $\mathbf{t}_u = (W, y_u, 0)$, $\mathbf{t}_v = (0, y_v, L)$.

The data are stored as three precomputed $N_b \times 32 \times 32$ float32 buffers, uploaded once to the GPU and accessed via 128 binding slots.

4.2 Gravity Library: 20-Bin NLGEOM with Normal Coupling

The gravity deformation is obtained from ANSYS MAPDL nonlinear static simulations (SHELL181 elements, NLGEOM ON, 20 tilt angles $\theta \in \{10^\circ, 14^\circ, \dots, 80^\circ\}$, spacing $\leq 4^\circ$). The 20-bin density was validated against a 5-bin prototype: the sparse bins produced systematic slope underestimation at off-bin angles (slope correlation dropping to 0.77–0.82), causing a “small-spot illusion.” At 20 bins, the linear interpolation residual is negligible ($R^2 \geq 0.96$, slope correlation ≥ 0.994).

Each bin stores a **3-plane format** ($w, \partial w / \partial u, \partial w / \partial v$) on the same 32×32 pixel-centered grid, packed into a single merged buffer of $20 \times 3 \times 1024$ floats (240 kB). The three-plane design is essential: both the height w and its slopes enter the surface tangent vectors, so the surface normal becomes

$$\mathbf{n} = -\text{normalize}(\mathbf{t}_u \times \mathbf{t}_v), \quad \mathbf{t}_u = (W, y_u^{\text{grav}} + y_u^{\text{bolt}}, 0), \quad \mathbf{t}_v = (0, y_v^{\text{grav}} + y_v^{\text{bolt}}, L) \quad (5)$$

If only the height were embedded (a 1-plane format), the gravity deformation would alter the ray intersection point but not the reflected direction—the gravity effect would be invisible to the optical loss at heliostat-to-receiver distances. The three-plane embedding ensures that gravity deformation contributes to the surface normal and therefore to the optical gradient.

4.3 Gradient Chain: Optical $\rightarrow$ Mechanical

The gradient of the S95 loss with respect to bolt height h_b traces through three domains:

$$\frac{dL}{dh_b} = \sum_{d \in \mathcal{D}} \sum_{p \in \text{pixels}} \left[\frac{\partial L}{\partial f_p} \cdot \left(\underbrace{\frac{\partial f_p}{\partial y} \cdot \phi_b}_{\text{height channel}} + \underbrace{\frac{\partial f_p}{\partial y_u} \cdot \frac{\partial \phi_b}{\partial u}}_{u\text{-slope channel}} + \underbrace{\frac{\partial f_p}{\partial y_v} \cdot \frac{\partial \phi_b}{\partial v}}_{v\text{-slope channel}} \right) \right] \quad (6)$$

The optical partials $\partial f_p / \partial (y, y_u, y_v)$ are computed by Slang’s `bwd_diff` through the double-refraction ray trac-

Table 3 TPS proxy vs. NLGEOM FEA validation (North 300 m, 35-bolt optimal).

Angle	RMS (mm)	R^2	Shape corr.	Flux corr.
29.5°	1.94–1.96	0.955	0.980	> 0.997
58.5°	2.04–2.05	0.952	0.980	> 0.997

ing code in `forward.slang`, accumulated per pixel, and stored in a 12 KB fixed-point tile ($\times 10^5$ scale, 32-bit unsigned integer, interlocked atomic addition). A reduction pass (`reduceSurfaceGradients`) converts the tile to a float3 surfaceGradient buffer, and the projection pass (`projectBoltGradients`) evaluates Eq. 6 using the pre-computed influence buffers.

This three-stage design (per-pixel fixed-point accumulation $\rightarrow$ cross-group float reduction $\rightarrow$ bolt projection) is motivated by a GPU-specific constraint: `InterlockedAdd` on float is not universally available (pre-Vulkan 1.2), and a naive global atomic on the bolt gradient would serialize all thread groups. The fixed-point tile trades one integer atomic per pixel for one float reduction per sun direction—a $50\times$ reduction in atomic operations for 7,850 pixels and 35 bolts.

4.4 Proxy Fidelity

Table 3 summarizes the TPS proxy validation against NLGEOM FEA reference solutions. The systematic deviation (RMS 2.0–3.3 mm, shape correlation 0.95–0.96) arises because TPS is a purely geometric RBF interpolation—it does not encode plate bending stiffness D , material properties (E, ν), or free-edge boundary conditions. For the differential effect $\partial w / \partial h_b$ (which is what the gradient chain uses), the proxy accuracy is sufficient, as confirmed by the per-pixel flux NRMSE of 1.7–1.8% and correlation > 0.996 with FEA ray-traced reference spots.

5 Gradient Audit: Deformation Analysis and Bounds

With the physics proxy embedded in the differentiable renderer, we can now use the framework to answer RQ1 and RQ2: *how large is the gravity penalty, what are its physical origins, and how much of it can bolt adjustment actually remove?*

5.1 Deformation Decomposition

decompose the per-angle gravity displacement field $w(\mathbf{r}; \theta)$ into three spectral bands via least-squares fitting:

- **Affine** (overall tilt + piston): $w_{\text{affine}} = a_1 x + a_2 z + a_3$.
- **Quadratic** (focus-like): $w_{\text{quad}} = b_1 x^2 + b_2 z^2 + b_3 xz$.
- **Higher-order** (inter-support sag): the residual after removing affine and quadratic components.

The affine component is *identically zero* at all angles: the 35 support bolts pin the plate’s mean plane, so there is no net tilt or piston to compensate. This has an important implication: panel-level adjustment methods (canting, aiming) that act exclusively on affine degrees of freedom cannot, in principle, touch the dominant gravity damage. The quadratic component is small (equivalent slope ≤ 0.6 mrad). The higher-order inter-support sag dominates: 6.36 mrad equivalent slope at 10° tilt, decreasing to 0.92 mrad at 80° .

5.2 Compensability: Projection onto the Bolt Subspace

To determine how much of this deformation can be compensated by bolt adjustment, we project the gravity slope field onto the subspace spanned by the 35 TPS influence functions. The projection is performed in *slope space* because the optical consequence is driven by normal errors, not height errors:

$$\text{removal_ratio}(\theta) = \frac{\|\nabla w_{\text{grav}}\|^2 - \min_{\mathbf{h}} \|\nabla w_{\text{grav}} - \sum_b h_b \nabla \phi_b\|^2}{\|\nabla w_{\text{grav}}\|^2} \quad (7)$$

The result: TPS influence functions can remove only **26–38%** of the slope variance of the higher-order sag component. The remaining 62–74% is **structurally irreducible** under the given 35-bolt (7×5 , 8% edge margin) support layout.

5.3 The Bounds Framework

We formalize this into a four-level bounds hierarchy that characterizes the optimization landscape:

- B^* (gravity-free optimum): S95 of the best surface achievable with gravity disabled—the ultimate lower bound.
- B_{ideal} (gravity-free ideal): S95 of the ideal elliptic paraboloid surface evaluated without gravity.
- B_{comp} : S95 using closed-form slope-space gravity compensation as bolt initialization.
- B_{naive} : S95 of the ideal elliptic surface evaluated with gravity enabled, without any compensation.

Table 4 shows these bounds for the four cardinal mirrors at 300 m from a 300 m tower (Delingha, 37.36°N). The key finding: **84–87% of the naive–ideal gap is structurally irreducible** for the East, South, and West mirrors. Closed-form compensation recovers only 10–14%; end-to-end optimization adds at most another 3–5%. The floor is a structural property of the support layout, not an artifact of the optimization setup.

5.4 Ablation Study: The Floor is Invariant

A nine-group ablation experiment varied five factors simultaneously: tanh-bounded vs. unbounded parameterization;

Table 4 Four-level bounds: S95 (m^2) at 300 m, 36 sun directions.

Bound	North	East	South	West
B^* (no-gravity opt. lower bound)	49.77	65.00	73.07	64.68
B_{ideal} (no-gravity LSQ)	51.32	65.68	73.51	65.60
B_{comp} (closed-form comp.)	51.75	76.71	94.90	76.83
B_{naive} (LSQ + gravity)	51.31	77.90	98.33	78.07
Struct. irreducible (%)	37%	86.9%	84.0%	86.0%

slope-space anchor strength $\lambda_s \in \{0, 10^3, 10^4, 10^5\}$; soft stroke wall on/off; bending energy λ_b on/off; and initialization (naive vs. compensated). The result: the East, South, and West mirrors’ final S95 values are **locked to 76.2, 94.3, and 76.2 m^2** respectively, with a range of $\leq 0.13 \text{ m}^2$ across all nine groups—within Monte Carlo noise. The structural floor is invariant to parameterization, regularization strength, constraint type, and initialization. This invariance constitutes the primary evidence that the floor is a *structural* property of the support layout.

From Surface Compensation to Structural Optimization

The conclusion that the structural floor is determined by the support layout, not by bolt stroke optimization, directly motivates making the layout itself an optimization variable.

6.1 The Cantilever Discovery

A density scan tested whether adding more bolts within the same 8% margin could lower the floor: 48 bolts (8×6), 49 bolts (7×7), 63 bolts (9×7). The result was unexpected: **+80% more bolts (from 35 to 63) reduced the gravity floor by only 4.2%**. Two competing plate-theory models (max-span⁴ and Navier simply-supported) both predicted substantially larger reductions, and were simultaneously falsified.

Spatial partitioning of the slope energy revealed the cause: **80–91%** of the gravity slope energy (across all angles) is concentrated in the *cantilevered edge flaps*—the region outside the outermost bolt row—not in the inter-bolt bays that additional bolts would subdivide. The cantilever is an unsupported free edge, and its deformation scales as $w \propto a^4$ (where a is the overhang distance) in thin-plate theory.

6.2 The Margin–Span Conservation Law

Reducing the edge margin m (the distance from the outermost bolt row to the plate edge, as a fraction of the half-width/half-length) simultaneously:

- **Reduces** the cantilever overhang $\propto m$, lowering the flap deformation $\propto m^3$;
- **Increases** the inter-bolt span (for a fixed number of bolts), raising the bay deformation $\propto (1 - m)^3$.

Table 5 Margin optimization: per-mirror annual S95 (m²), 334 sun directions.

Margin	North	East	South	West	Total
0.08 (baseline)	72.75	101.59	116.57	93.28	384.2
0.05 (optimal)	54.06	73.23	81.28	73.09	281.7
Dividend (%)	−25.7	−27.9	−30.3	−21.6	−26.7

The opposing scaling implies an **interior optimum** for the margin at a given bolt density. This “margin–span conservation law” predicts that margin optimization can reduce the gravity floor *with zero additional bolts*—a free structural dividend.

6.3 Margin Optimization Results

To test this prediction, we scanned the margin across five values $m \in \{0.08, 0.06, 0.05, 0.04, 0.03\}$ by regenerating the ANSYS gravity bins at each margin and running the end-to-end optimization pipeline (100 iterations, 110 sun directions, four mirrors). The gravity field for each margin was obtained from fresh NLGEOM FEA solves with fine substeps (NSUBST 50, 500, 50) to ensure quasi-static smooth branch following.

The optimum occurs at $m^* = 0.04$ – 0.05 , with point estimate $m^* = 0.05$. At 334 sun directions, the annual-averaged S95 for the four 300 m mirrors totals:

$$S95_{\text{total}}(m=0.05) = 281.7 \text{ m}^2 \quad \text{vs.} \quad S95_{\text{total}}(m=0.08) = 384.2 \text{ m}^2 \quad (8)$$

a **26.7% reduction** achieved purely by moving the bolt rows closer to the plate edges—no additional bolts, no material change, no manufacturing cost increase.

6.4 Methodological Note: NLGEOM Substep Requirements

During this investigation, we encountered a systematic artifact in per-angle independent NLGEOM solves with coarse substeps (ANSYS NSUBST 1, 10, 1). At tilt angles near 46°, the snap-through bistability of the thin plate causes the independent cold-start solve to randomly select one of two stable branches. This produces gravity bins with *sign-flipped* deformation fields at high tilt angles, rendering entire margin-scan results invalid. The issue was resolved by using fine substeps (NSUBST 50, 500, 50) to guarantee quasi-static smooth branch following, which shifted the point estimate from $m^*=0.04$ to $m^*=0.05$. This finding constitutes a methodological contribution: *any multi-angle parameterized model relying on per-angle independent FEA solves must verify branch continuity with fine substeps or numerical continuation.*

Fig. 1 Annual gravity penalty map: S95 inflation ratio across 20 mirrors (334 sun directions). North mirrors are nearly immune; South near-field is the worst case at $1.539\times$.

7 Experiments

7.1 Experimental Setup

- **Hardware:** NVIDIA RTX 4070 SUPER 12 GB (desktop), RTX 4060 8 GB (laptop). Vulkan 1.3, Slang compiler.
- **Heliostat:** 12.84×9.45 m glass mirror, thickness 4 mm, $n = 1.523$, 35 support bolts (7×5 , margin 8% unless otherwise noted). Plate bending stiffness $D = 392 \text{ N}\cdot\text{m}$, gravity load $q = 98.1 \text{ N/m}^2$.
- **Receiver:** Cylindrical, radius 10 m, height 20 m, discretized at 157×50 pixels.
- **Sun model:** Buie [19] CSR= 0.01, DNI= 1000 W/m², slope error $\sigma_s = 1 \text{ mrad}$.
- **Sun directions:** 36 directions for fast iteration, 110 (paper-minimal), 334 (balanced annual), all symmetric about true solar noon (azimuth= 180°, eliminating the equation-of-time asymmetry).
- **Optimizer:** Adam ($\beta_1=0.9$, $\beta_2=0.999$), learning rate 4×10^{-4} constant, 100–200 iterations.
- **Site:** Delingha, Qinghai, China (37.36°N, 97.29°E), tower height 300 m.

7.2 RQ1: The Real Gravity Penalty Map

Figure 1 shows the annual S95 inflation ratio $S95_{\text{naive}}/S95_{\text{ideal}}$ across 20 mirrors (five slant ranges 150/300/600/900/1200 m $\times$ four cardinal directions N/E/S/W, 334 sun directions). The gravity penalty is **spatially strongly non-uniform**: North mirrors are nearly immune across all distances (ratio 1.001–1.02 $\times$), while the worst case is the South 150 m mirror at $1.539\times$.

The non-uniformity has a purely geometric explanation. The annual tilt angle θ (angle between plate normal and vertical) distribution for each mirror position is determined by site latitude, tower geometry, and mirror coordinates alone—no optical simulation required. At Delingha’s latitude:

- **North** 300 m: $\theta \in [36^\circ, 65^\circ]$, median 51° . 35% of annual time falls in the 40–50° band, near the $\sim 46^\circ$ NLGEOM zero-crossing where the gravity-induced sag is minimal.
- **South** 300 m: $\theta \in [0^\circ, 57^\circ]$, with 24% of time at $\theta < 20^\circ$, where the gravity bending moment $q \cos \theta$ is maximal.
- **East/West** 300 m: cross the 46° sign-change point,

carrying an irreducible θ -variation component of 2.5–2.9 mrad that no fixed bolt setting can compensate.

This tilt-angle distribution model predicts the measured penalty map with Pearson $r = 0.913$ (Spearman $\rho = 0.922$), transforming the empirical observation into a **design rule**: which heliostats in a field are worth structural compensation can be determined from latitude and tower geometry alone.

7.3 RQ2: The Irreducible Floor

§5 presented the quantitative bounds. Here we highlight the experimental evidence for the floor’s invariance. The nine-group ablation (Table 4) showed that the East, South, and West mirrors’ final S95 is locked to ~ 76.2 , 94.3, and 76.2 m² respectively, with range ≤ 0.13 m². Neither the parameterization (bounded vs. unbounded), nor regularization strength (0 – 10^5), nor initialization (naive vs. compensated) could move the East/West mirrors below 76.2 or the South mirror below 94.3.

The path forward is structural. With fixed $N_b = 35$ and fixed margin $m = 8\%$, bolt stroke adjustment—however sophisticated—can recover at most ~ 13 – 16% of the gravity penalty. The remaining 84–87% requires changing the support layout itself.

7.4 RQ3: Structural Optimization

§6 detailed the margin optimization. Here we supplement two additional structural findings.

Bolt density at optimal margin. At the optimal margin $m^* = 0.05$, a density scan across layouts 7×5 (35), 9×7 (63), 9×8 (72), and 11×9 (99 bolts) reveals that 9×7 (63 bolts, +80%) reduces S95 by a further 10–14% beyond the margin dividend. However, 11×9 (99 bolts, +183%) shows *degradation* on East and West mirrors—the excessive degrees of freedom cause the optimizer to fall into inferior local minima. The sweet spot emerges as a “density $\times$ aspect-ratio” surface with 10×7 (70 bolts) producing the overall best results.

Position–height joint optimization. To test whether per-bolt position freedom beyond uniform grids could yield further gains, we implemented TPS adjoint position sensitivity and a bold top-K step-ladder protocol. The result: across four initial grid topologies and two mirror orientations, per-bolt position moves of only 1–3 cm (from step sizes up to 30 cm offered) produce S95 improvements of **0.5–0.9%** before hitting a wall. **Grid topology choice dominates position micro-tuning.** No asymmetric layouts emerged, confirming that the uniform-grid optimum is near-global under per-bolt freedom.

Table 6 Per-iteration timing breakdown (RTX 4060 Laptop, 36 sun directions, single mirror).

Stage	Time (ms)
Mechanical forward (surface build)	0.3
Ray tracing forward (per sun)	18.2
S95 binary search	0.1
Loss + ray tracing backward (per sun)	26.5
Mechanical backward (reduce + project)	0.5
Adam update	< 0.1
Total per sun	45.6
Total per iteration (36 suns)	1,640

7.5 Timing and Scalability

Table 6 reports per-iteration timing. The dominant cost is the ray tracing forward and backward passes, which scale with pixel count ($157 \times 50 = 7,850$) and sun direction count. The S95 search and mechanical passes are negligible (< 1 ms each). A 200-iteration optimization with 36 sun directions completes in ~ 7 minutes for a single mirror and ~ 20 minutes for four mirrors.

8 Discussion

8.1 What Did the Physics Proxy Actually Do?

The central question motivating this work—what does embedding a physics proxy into a differentiable renderer actually achieve?—can now be answered with quantitative evidence:

- (1) **Diagnostic role:** The proxy enabled the gradient audit methodology that certified the structural floor and proved its invariance to all optimizer and regularization choices. The finding that “84–87% of the gap is structural” was independently verified by FEA spot-checks.
- (2) **Engineering role:** The proxy carried the deformation field through the differentiable loop fast enough to make structural optimization practical—replacing per-iteration FEA solves (3 min each) with GPU-resident μ s evaluations.
- (3) **Explanatory role:** The proxy translated the empirical observation “bolt density shows diminishing returns” [15, 16] into a mechanical mechanism—the cantilever-dominant slope energy that the margin–span conservation law formalizes.

The proxy’s real value lies not in per-mirror compensation (where it can only recover 13–16%), but in *driving structural design decisions* (where the margin dividend of 26.7% dwarfs the compensation ceiling).

8.2 Limitations

TPS linearity. The TPS proxy assumes linear superposition of bolt effects, which the NLGEOM FEA validates with

shape correlation 0.95–0.96. Residual nonlinearity limits the proxy’s accuracy for absolute deformation prediction but is adequate for the differential effect $\partial w / \partial h_b$ used in gradient computation.

Single-site, single-load. Our experiments consider one site (Delingha) and one dominant load (gravity). Wind-induced deformation, thermal gradients, and manufacturing tolerances are not modeled. Incorporating these loads into the differentiable pipeline would require additional FEA simulations but no architectural changes.

Steel vs. glass. Real heliostats use steel-backed glass modules (Kesheng V3: 5 mm steel, rail spacing 0.37 m). We verified via FEA that steel and glass produce near-identical deformation shapes (cosine similarity > 0.996) with a pure-scaling factor $\alpha \approx 1.01$, meaning the structural conclusions (margin optimum, density surface) transfer qualitatively. However, the rail-stiffened floor is $\sim 500\times$ stiffer than the bare plate, so the absolute S95 numbers require recalibration for production hardware.

8.3 Extensions

Full-field scaling. The per-mirror S95 results can be converted to annual field-level energy yield by convolving with the mirror population distribution and a receiver efficiency model.

Aiming co-optimization. Joint optimization of bolt strokes and per-mirror aiming points (2 additional DOF per mirror) would close the last open loop in the optical chain.

Branch engineering. The NLGEOM bistability at $\sim 46^\circ$ (§6.4) is currently treated as an artifact to be avoided. However, the flipped branch is *optically better* (lower gravity sag) at high tilt angles. Exploiting this bistability through controlled snap-through actuation is a speculative but potentially high-impact extension.

9 Conclusion

We have presented a GPU-native differentiable Monte Carlo ray tracing system that embeds finite-element-derived physics proxies to enable end-to-end gradient-based optimization of heliostat surfaces across structural and optical domains. Three main findings structure the work:

- (1) **The gravity penalty is real, large, and predictable.** Gravity-induced S95 inflation reaches $1.539\times$ in the worst case and is strongly non-uniform across the field. The non-uniformity can be predicted from site geometry alone, transforming an empirical observation into a design rule.
- (2) **The structural floor can be bounded and certified.** Under a fixed 35-bolt layout, only 26–38% of the gravity

slope variance is reachable by bolt adjustment; 84–87% of the optical penalty is structurally irreducible. This floor is invariant to optimizer, regularization, and initialization—a gradient audit methodology with applicability to physics-embedded differentiable systems.

- (3) **The floor can be moved by structural optimization.** Discovering that 80–90% of gravity energy concentrates in cantilevered edges leads to a margin–span conservation law and a zero-cost 26.7% S95 reduction through margin optimization alone, verified by full ANSYS truth re-solve.

The framework transforms gravity compensation from an empirical, per-mirror adjustment process into a bounded, certifiable, and fully automated design workflow. The gradient audit methodology—decomposing a physics proxy’s contribution into reachable and irreducible subspaces—is not specific to heliostats and may serve as a template for other domains where physical proxies are embedded in differentiable rendering pipelines.

Appendix A: ANSYS Automation and Coordinate Conventions

The ANSYS MAPDL batch pipeline automates per-angle NLGEOM static solves with SHELL181 shell elements and 35 bolt-node translation constraints. The plate local coordinate system is defined by the plate normal $(0, \cos \theta, +\sin \theta)$ in global coordinates, with the displacement extraction $w = u_y \cos \theta + u_z \sin \theta$.

Appendix B: Sun Direction Sampling

Sun directions are generated symmetric about *true* solar noon (azimuth = 180°), eliminating the equation-of-time asymmetry that would bias morning/afternoon sampling in mean-solar-noon schemes. Three presets are available: *paper* ($12 \text{ months} \times 1 \text{ day} \times 13 \text{ hours}$, ~ 110 directions), *balanced* ($12 \times 3 \times 13$, ~ 334 directions), and *dense* ($12 \times 14 \times 13$, $\sim 1,556$ directions).

Availability of data and materials

The source code and pre-generated proxy data are available at https://github.com/xxx/bezier_opt_desktop. The ANSYS FEA pipeline requires a commercial ANSYS MAPDL license; pre-generated gravity bins are included in the repository.

Author contributions

All authors contributed to the system design, experiments, and manuscript preparation. All authors have approved the final manuscript.

Acknowledgements

This work was supported by [TODO: funding information].

Declaration of competing interest

The authors have no competing interests to declare that are relevant to the content of this article.

References

- [1] Nimier-David, M.; Vicini, D.; Zeltner, T.; Jakob, W. Mitsuba 2: A retargetable forward and inverse renderer. *ACM Transactions on Graphics* Vol. 38, No. 6, Article No. 203, 2019.
- [2] Jakob, W.; Speierer, S.; Roussel, N.; Nimier-David, M.; Vicini, D.; Zeltner, T.; Nicolet, B.; Crespo, M.; Leroy, V.; Zhang, Z. Mitsuba 3 renderer, 2022. Available at <https://mitsuba-renderer.org>.
- [3] Yan, Y.; Zhang, Z.; Jakob, W. ZeroGrads: Learned optimization of production assets via differentiable rendering. *ACM Transactions on Graphics* Vol. 43, No. 4, 2024.
- [4] Zhou, K.; Chen, W.; Yu, J. Computational caustic design for surface light sources. *IEEE Transactions on Visualization and Computer Graphics* Vol. 32, No. 2, 2026.
- [5] Jakob, W. Differentiable rendering. In: *ACM SIGGRAPH 2022 Courses*, 2022.
- [6] He, Y.; Foley, T.; Tatarchuk, N.; Fatahalian, K. Slang: A language for real-time GPU shading and differentiation. *ACM Transactions on Graphics* Vol. 41, No. 6, 2023.
- [7] (TBC) A novel heliostat aiming optimization framework via differentiable Monte Carlo ray tracing for solar power tower. *Applied Energy*, 2025.
- [8] (TBC) The optimization of heliostat paraboloid canting via differentiable ray tracing. *Solar Energy*, 2025.
- [9] Pargmann, M.; Ertl, J.; Kesselheim, S.; Göttische, J.; Schorn, C.; Klink, S. Automatic heliostat learning for in situ concentrating solar power plant metrology with differentiable ray tracing. *Nature Communications* Vol. 15, Article No. 6997, 2024.
- [10] (TBC) Inverse deep learning raytracing for heliostat surface prediction. *Solar Energy*, 2025.
- [11] (TBC) Scalable heliostat surface predictions from focal spots: Sim-to-Real transfer of inverse deep learning raytracing. *Solar Energy*, 2025.
- [12] Karniadakis, G. E.; Kevrekidis, I. G.; Lu, L.; Perdikaris, P.; Wang, S.; Yang, L. Physics-informed machine learning. *Nature Reviews Physics* Vol. 3, 422–440, 2021.
- [13] (TBC) Differentiable automatic structural optimization using graph deep learning. *Advanced Engineering Informatics*, 2024.
- [14] (TBC) Digital twin of wind farms via physics-informed deep learning. *Energy Conversion and Management*, 2023.
- [15] Li, B.; Yan, J.; Zhou, W.; Peng, Y. Influence of service load and structural parameters on optical accuracy of solar tower heliostat (in Chinese). *Acta Optica Sinica* Vol. 44, No. 6, Article No. 0623001, 2024.
- [16] Yan, J.; Song, T.; Peng, Y.; Zhou, W. Optical accuracy study of pentagonal tower solar heliostat based on support bolt jacking molding (in Chinese). *Acta Optica Sinica* Vol. 45, No. 6, Article No. 0608001, 2025.
- [17] Yang, (TBC) et al. A coupled structural-optical analysis of a novel umbrella heliostat. *Solar Energy*, 2022.
- [18] Thalange, (TBC) et al. Design, optimization and optical performance study of tripod heliostat for solar power tower plant. *Energy* Vol. 135, 610–624, 2017.
- [19] Buie, D.; Monger, A. G.; Dey, C. J. Sunshape distributions for terrestrial solar simulations. *Solar Energy* Vol. 74, No. 2, 113–122, 2003.
- [20] Collado, F. J.; Guallar, J. A review of optimized design layouts for solar power tower plants with Campo code. *Renewable and Sustainable Energy Reviews* Vol. 20, 142–154, 2013.
- [21] Lin, R.; (TBC) et al. Real-time radiative flux density distribution simulation via data-driven neural convolution model (Fast-NCM). Preprint, 2026.
- [22] Ho, C. K.; Khalsa, S. S. A flux density distribution model based on convolution of Gaussian and error functions (HFLCAL). *Journal of Solar Energy Engineering* Vol. 134, No. 1, 2012.
- [23] Wendelin, T. SolTRACE: A new optical modeling tool for concentrating solar optics. In: *Proceedings of the ASME International Solar Energy Conference*, 2003.
- [24] Blanco, M. J.; et al. Tonatiuh: An open source Monte Carlo ray tracer for solar concentrating systems. *Solar Energy*, 2010.
- [25] (TBC) Kesheng V3 heliostat specifications. Private communication, 2026.

Author biography

First A. Author [TODO: author photo and biography]